

编译原理研讨课第三次实验报告

第 13 组： 李浩宇 周熙博 武治行

一、任务目标

在已完成 CACT 语言的词法分析、语法分析、语义分析及中间代码生成的基础上，实现从中间代码到 RISC-V 汇编代码的转换，构建完整的编译器后端流程。

二、实验环境

1 工具链：

- 使用 riscv64-unknown-linux-gnu-gcc 交叉编译器
- 通过 QEMU 模拟 RISC-V 环境运行可执行文件

2 库文件：

- 预编译运行时库 libmylib.a（由 runtime.c 生成，含 getint 等内置函数）

3 编译执行流程：

```
./compiler ../test/functional/$num.cact
```

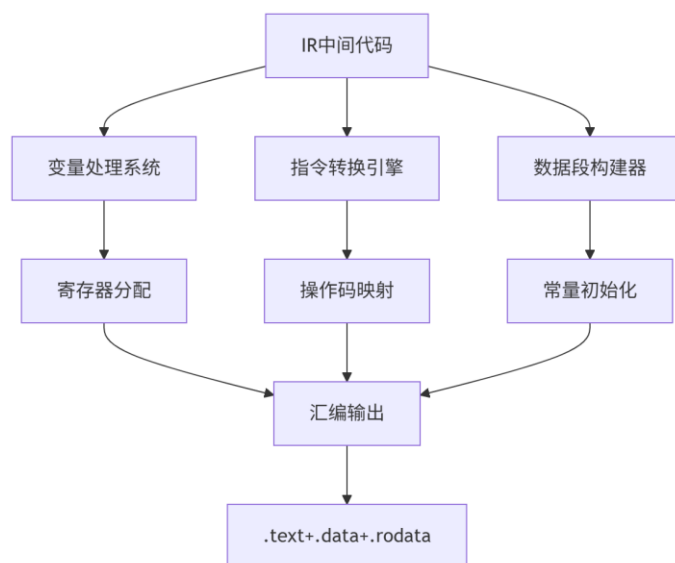
```
riscv64-unknown-linux-gnu-gcc -c $num.S -o $num.o
```

```
riscv64-unknown-linux-gnu-gcc $num.o -L. -lmylib -o $num -static
```

```
qemu-riscv64 ./ $num; echo $?
```

三、设计思路

1.整体上采用分层架构实现 IR 到 RISC-V 汇编的转换：



2.核心模块功能设计

2.1 变量存取系统

- 三级变量分类

变量类型	前缀	存取策略
全局变量	g_	la 加载全局地址+偏移
局部变量	l_	s0 基址+栈偏移

变量类型	前缀	存取策略
临时变量	t_	寄存器直接操作
立即数	imm_	.rodata 段标签引用

- 类型敏感指令选择

```
string select_insn(IR_OP op, Basetype type) {
    if (type == BTY_FLOAT) {
        return float_ops[op]; // fadd.s/fsub.s 等
    } else {
        return int_ops[op];    // add/sub 等
    }
}
```

2.2 数据段初始化机制

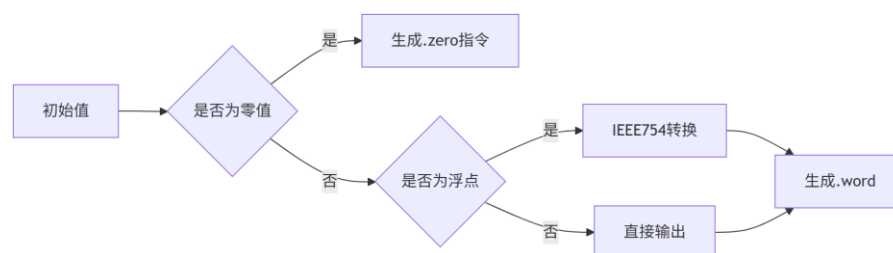
- 常量处理策略

A.浮点数：转换为 IEEE754 十六进制格式

B.数组：展开为连续存储块

C.零值块：.zero 指令压缩存储

- 智能初始化流程



2.3 函数调用约定实现

- 栈帧管理模型

高地址

```

+-----+
| 返回值地址(ra) | ← frame_size-8
+-----+
| 帧指针(s0)      | ← frame_size-16
+-----+
| 参数 1-8        | ← 16-80
+-----+
| 局部变量区      |
+-----+
| 对齐填充        | ← 16 字节对齐
+-----+ ← sp

```

低地址

四、代码实现

1. 核心类结构

```
class RiscvGen {private:
    vector<string> ASM_array;           // 指令缓存
    vector<string> Const_Stack;        // 常量池
    int frame_size;                     // 当前栈帧大小
    bool rescan;                        // 二次扫描标志
    int rescan_loc;                     // 函数起始位置
public:
    void Gen_All();                     // 主生成函数
    void var_to_reg(string var, string reg); // 变量加载
    void reg_to_var(string reg, string var); // 结果写回
    void Gen_FuncBegin(IR &irc);        // 函数入口
    void Gen_FuncEnd(IR &irc);          // 函数出口
    void stack_initval(string initval, Basetype basety); // 初始化
    // ... 其他成员函数
};
```

2. 双阶段扫描算法

Gen_All 核心逻辑:

```
void RiscvGen::Gen_All() {
    // 首次扫描: 元数据收集
    for (int i = 0; i < IRC_array.size(); i++) {
        IR &irc = IRC_array[i];
        if (!rescan) {
            // 函数栈帧分析
            if (irc.IRop == IR_FUNC_BEGIN) {
                frame_size = 16;
                rescan_loc = i;
            }
            else if (irc.IRop == IR_PARAM) {
                frame_size += 8; // 参数空间
            }
            // ... 其他元数据收集
        }
    }

    // 二次扫描: 代码生成
    for (int i = rescan_loc; i < IRC_array.size(); i++) {
        switch (irc.IRop) {
            case IR_FUNC_BEGIN:
                Gen_FuncBegin(irc);
                break;
```

```

        case IR_ADD: case IR_FADD:
            Gen_Operation(irc);
            break;
        // ... 其他指令处理
    }
}
}

```

3. 关键函数实现

3.1 变量加载系统

```

void RiscvGen::var_to_reg(string var, string reg) {
    if (var[0] == 'g') { // 全局变量
        string label = "G_" + var.substr(2);
        emit("la t0, " + label);
        emit(load_insn(type) + " " + reg + ", 0(t0)");
    }
    else if (var[0] == 'l') { // 局部变量
        int offset = stoi(var.substr(2));
        emit(load_insn(type) + " " + reg + ", " + to_string(offset) + "(s0)");
    }
    // ... 其他类型处理
}

```

3.2 函数入口生成

```

void RiscvGen::Gen_FuncBegin(IR &irc) {
    string func_name = irc.result.substr(1);

    // 函数头
    emit(".text");
    emit(".globl " + func_name);
    emit(func_name + ":"");

    // 栈帧建立
    emit("addi sp, sp, -" + to_string(frame_size));
    emit("sd ra, " + to_string(frame_size-8) + "(sp)");
    emit("sd s0, " + to_string(frame_size-16) + "(sp)");
    emit("addi s0, sp, " + to_string(frame_size));

    // 参数保存
    for (int i = 0; i < param_count; i++) {
        string reg = (i < 8) ? "a" + to_string(i) : "t0";
        emit("sd " + reg + ", " + to_string(16 + i*8) + "(s0)");
    }
}

```

3.3 浮点常量处理

```

void RiscvGen::handle_float_init(string val) {

```

```

float fval = stof(val);
uint32_t binary = *reinterpret_cast<uint32_t*>(&fval);

// 生成浮点常量标签
string label = "FLOAT_CONST_" + to_string(const_count++);
emit(".section .rodata");
emit(".align 2");
emit(label + ":");
emit(".word 0x" + to_hex(binary));

// 加载到寄存器
emit("la t0, " + label);
emit("lw t1, 0(t0)");
emit("fmv.w.x " + reg + ", t1");
}

4. 数据段生成优化
void RiscvGen::stack_initval(string initval, Basetype basety) {
    vector<string> vals = split(initval, ',');
    size_t base_size = type_size(basety);

    for (auto &val : vals) {
        if (val == "0") {
            zero_count += base_size;
        } else {
            if (zero_count > 0) {
                Const_Stack.push_back(".zero " + to_string(zero_count));
                zero_count = 0;
            }
            if (basety == BTY_FLOAT) {
                handle_float_init(val);
            } else {
                Const_Stack.push_back(".word " + val);
            }
        }
    }
}

// 尾部零值处理
if (zero_count > 0) {
    Const_Stack.push_back(".zero " + to_string(zero_count));
}
}

```

五、实验结果总结

本实验中，我们成功构建了从中间代码到 RISC-V 汇编的转换流程。整个 RISC-V 变量生成过

程以作用域和类型为核心，配合中间变量命名规范，对变量进行统一抽象编码处理。汇编生成过程中充分考虑了 RISC-V 的调用约定、类型对应的指令选择，并为数组和指针提供了灵活的偏移支持，保证了变量访问在汇编层面的一致性和正确性。这个过程中 `stack_initval`、`var_to_reg`、`reg_to_var` 作为变量处理的基础函数，是连接语义分析结果与汇编生成之间的重要桥梁。编译运行所有 functional 的 32 个样例都成功通过。编译好的汇编文件、目标文件和可执行文件见于目录 `functional_asm` 下。

小组分工：

1. 李浩宇：

- 参与核心代码生成模块的编写（如指令选择、控制流翻译等）
- 负责功能测试与调试（functional 测试样例）
- 实验报告的撰写与整合

2. 周熙博：

- 实验环境搭建与配置（交叉编译工具链、QEMU）
- 实现内置 IO 库（`runtime.c` 的编写与集成）
- 参与 bug 修复与系统测试

3. 武治行：

- 设计中间代码到汇编的转换策略（包括变量存储、函数调用约定等）
- 研究 RISC-V 汇编规范及 ABI，实现栈帧管理、参数传递等关键模块
- 伪指令的处理与优化